

U-Net Lane/Driveable area Detection

Generated by Doxygen 1.9.1

1 Namespace Index	1
1.1 Namespace List	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Namespace Documentation	9
5.1 dataset Namespace Reference	9
5.1.1 Variable Documentation	9
5.1.1.1 train_transforms	9
5.2 model Namespace Reference	9
5.3 train Namespace Reference	10
5.3.1 Function Documentation	10
5.3.1.1 check_accuracy()	11
5.3.1.2 check_masks()	12
5.3.1.3 load_checkpoint()	13
5.3.1.4 main()	13
5.3.1.5 save_predictions_as_imgs()	14
5.3.1.6 train_fn()	15
5.3.2 Variable Documentation	16
5.3.2.1 batch_size	16
5.3.2.2 device	16
5.3.2.3 exist_ok	16
5.3.2.4 filename	16
5.3.2.5 format	16
5.3.2.6 learning_rate	17
5.3.2.7 level	17
5.3.2.8 LOAD_MODEL	17
5.3.2.9 num_epochs	17
5.3.2.10 patience	17
5.3.2.11 save_dir	17
5.3.2.12 saved_images_dir	17
6 Class Documentation	19
6.1 train.DiceLoss Class Reference	19
6.1.1 Detailed Description	20
6.1.2 Constructor & Destructor Documentation	20
6.1.2.1 __init__()	20

6.1.3 Member Function Documentation	20
6.1.3.1 forward()	20
6.1.4 Member Data Documentation	21
6.1.4.1 smooth	21
6.2 model.DoubleConv Class Reference	21
6.2.1 Detailed Description	22
6.2.2 Constructor & Destructor Documentation	22
6.2.2.1 __init__()	22
6.2.3 Member Function Documentation	23
6.2.3.1 forward()	23
6.2.4 Member Data Documentation	23
6.2.4.1 conv	23
6.3 train.FocalLoss Class Reference	23
6.3.1 Detailed Description	24
6.3.2 Constructor & Destructor Documentation	25
6.3.2.1 __init__()	25
6.3.3 Member Function Documentation	25
6.3.3.1 forward()	25
6.3.4 Member Data Documentation	25
6.3.4.1 alpha	26
6.3.4.2 gamma	26
6.3.4.3 reduction	26
6.4 train.FocalLoss_w Class Reference	26
6.4.1 Detailed Description	27
6.4.2 Constructor & Destructor Documentation	27
6.4.2.1 __init__()	27
6.4.3 Member Function Documentation	27
6.4.3.1 forward()	27
6.4.4 Member Data Documentation	28
6.4.4.1 alpha	28
6.4.4.2 gamma	28
6.4.4.3 reduction	28
6.5 dataset.MultiClassLaneDataset Class Reference	29
6.5.1 Detailed Description	30
6.5.2 Constructor & Destructor Documentation	30
6.5.2.1 __init__()	30
6.5.3 Member Function Documentation	30
6.5.3.1 __getitem__()	30
6.5.3.2 __len__()	31
6.5.4 Member Data Documentation	31
6.5.4.1 base_transform	31
6.5.4.2 image_dir	31

6.5.4.3 images	31
6.5.4.4 mask_dir	31
6.5.4.5 num_augmentations	32
6.5.4.6 total_samples	32
6.5.4.7 transform	32
6.6 model.UNET Class Reference	32
6.6.1 Detailed Description	33
6.6.2 Constructor & Destructor Documentation	33
6.6.2.1 __init__()	33
6.6.3 Member Function Documentation	33
6.6.3.1 forward()	34
6.6.3.2 initialize_weights()	34
6.6.4 Member Data Documentation	34
6.6.4.1 bottleneck	34
6.6.4.2 downs	34
6.6.4.3 final_conv	35
6.6.4.4 pool	35
6.6.4.5 ups	35
7 File Documentation	37
7.1 dataset.py File Reference	37
7.2 model.py File Reference	37
7.3 train.py File Reference	38
Index	39

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

dataset	9
model	9
train	10

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

nn.Module	
model.DoubleConv	21
model.UNET	32
train.DiceLoss	19
train.FocalLoss	23
train.FocalLoss_w	26
Dataset	
dataset.MultiClassLaneDataset	29

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

train.DiceLoss	Dice Loss for multiclass segmentation	19
model.DoubleConv	A double convolution block for the U-Net architecture	21
train.FocalLoss	Focal Loss for multiclass segmentation	23
train.FocalLoss_w	Focal Loss with class weights for multiclass segmentation	26
dataset.MultiClassLaneDataset	A custom dataset for multiclass lane detection	29
model.UNET	U-Net architecture for semantic segmentation	32

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

dataset.py	37
model.py	37
train.py	38

Chapter 5

Namespace Documentation

5.1 dataset Namespace Reference

Classes

- class [MultiClassLaneDataset](#)
A custom dataset for multiclass lane detection.

Variables

- [train_transforms](#)

5.1.1 Variable Documentation

5.1.1.1 train_transforms

`dataset.train_transforms`

Initial value:

```
1 = A.Compose([
2     A.Resize(height=144, width=256, interpolation=cv2.INTER_NEAREST),
3     A.HorizontalFlip(p=0.5),  #!< Randomly flip images horizontally.
4     A.RandomBrightnessContrast(p=0.5),  #!< Adjust brightness and contrast randomly.
5     A.RandomGamma(p=0.5),  #!< Adjust gamma to enhance lane lines.
6     A.Normalize(mean=[0.41, 0.39, 0.42], std=[0.15, 0.14, 0.15], max_pixel_value=255.0),
7     ToTensorV2(),
8 ], additional_targets={'mask': 'mask'})
```

5.2 model Namespace Reference

Classes

- class [DoubleConv](#)
A double convolution block for the U-Net architecture.
- class [UNET](#)
U-Net architecture for semantic segmentation.

5.3 train Namespace Reference

Classes

- class [FocalLoss_w](#)
Focal Loss with class weights for multiclass segmentation.
- class [FocalLoss](#)
Focal Loss for multiclass segmentation.
- class [DiceLoss](#)
Dice Loss for multiclass segmentation.

Functions

- def [check_masks](#) (image_dir, mask_dir, dataset_type="train")
Checks for missing image-mask pairs in the dataset.
- def [save_predictions_as_imgs](#) (loader, model, epoch, folder="saved_images/", device="cuda")
Saves model predictions as images alongside input images and ground truth masks.
- def [train_fn](#) (loader, model, optimizer, loss_fn_focal, loss_fn_dice, scaler, epoch)
Trains the model for one epoch.
- def [check_accuracy](#) (loader, model, loss_fn_focal, loss_fn_dice, [device](#))
Evaluates the model on the validation dataset.
- def [load_checkpoint](#) (checkpoint, model)
Loads a saved model checkpoint.
- def [main](#) ()
Main training function.

Variables

- [device](#) = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
- int [batch_size](#) = 8
- int [num_epochs](#) = 1
- int [learning_rate](#) = 1e-3
- string [save_dir](#) = 'checkpoints'
- string [saved_images_dir](#) = 'saved_images'
- bool [LOAD_MODEL](#) = False
- int [patience](#) = 5
- [exist_ok](#)
- [filename](#)
- [level](#)
- [format](#)

5.3.1 Function Documentation

5.3.1.1 `check_accuracy()`

```
def train.check_accuracy (  
    loader,  
    model,  
    loss_fn_focal,  
    loss_fn_dice,  
    device )
```

Evaluates the model on the validation dataset.

Computes the validation loss and mean IoU for the given dataset.

Parameters

<i>loader</i>	(DataLoader): DataLoader for the validation dataset.
<i>model</i>	(nn.Module): The U-Net model to evaluate.
<i>loss_fn_focal</i>	(nn.Module): Focal loss function.
<i>loss_fn_dice</i>	(nn.Module): Dice loss function.
<i>device</i>	(str): Device to run the model on.

Returns

tuple: Mean validation loss and mean IoU.

Here is the caller graph for this function:

**5.3.1.2 check_masks()**

```
def train.check_masks (
    image_dir,
    mask_dir,
    dataset_type = "train" )
```

Checks for missing image-mask pairs in the dataset.

Parameters

<i>image_dir</i>	(str): Directory containing input images.
<i>mask_dir</i>	(str): Directory containing ground truth masks.
<i>dataset_type</i>	(str, optional): Type of dataset ('train' or 'val', default: 'train').

Returns

list: List of error messages for mismatched image-mask pairs.

Here is the caller graph for this function:

**5.3.1.3 load_checkpoint()**

```
def train.load_checkpoint (
    checkpoint,
    model )
```

Loads a saved model checkpoint.

Parameters

<i>checkpoint</i>	(dict): Checkpoint dictionary containing model state.
<i>model</i>	(nn.Module): The U-Net model to load the state into.

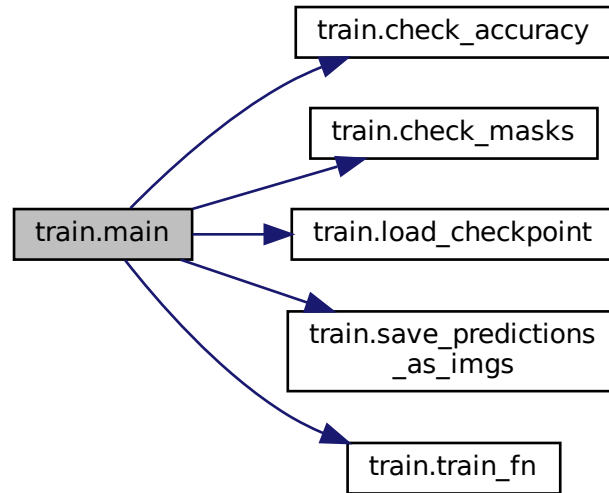
Here is the caller graph for this function:

**5.3.1.4 main()**

```
def train.main ( )
```

Main training function.

Orchestrates the training process, including dataset loading, model training, validation, and checkpoint saving. Here is the call graph for this function:



5.3.1.5 save_predictions_as_imgs()

```
def train.save_predictions_as_imgs (
    loader,
    model,
    epoch,
    folder = "saved_images/",
    device = "cuda" )
```

Saves model predictions as images alongside input images and ground truth masks.

Generates side-by-side images of the input, ground truth mask, and predicted mask for visualization.

Parameters

<i>loader</i>	(DataLoader): DataLoader for the validation dataset.
<i>model</i>	(nn.Module): The U-Net model to evaluate.
<i>epoch</i>	(int): Current epoch number.
<i>folder</i>	(str, optional): Directory to save images (default: 'saved_images/').
<i>device</i>	(str, optional): Device to run the model on (default: 'cuda').

Here is the caller graph for this function:



5.3.1.6 train_fn()

```
def train.train_fn (
    loader,
    model,
    optimizer,
    loss_fn_focal,
    loss_fn_dice,
    scaler,
    epoch )
```

Trains the model for one epoch.

Performs a single training epoch, computing combined Focal and Dice losses, and updating model parameters.

Parameters

<i>loader</i>	(DataLoader): DataLoader for the training dataset.
<i>model</i>	(nn.Module): The U-Net model to train.
<i>optimizer</i>	(optim.Optimizer): Optimizer for updating model parameters.
<i>loss_fn_focal</i>	(nn.Module): Focal loss function.
<i>loss_fn_dice</i>	(nn.Module): Dice loss function.
<i>scaler</i>	(GradScaler): Gradient scaler for mixed precision training.
<i>epoch</i>	(int): Current epoch number.

Returns

tuple: Mean training loss and mean IoU for the epoch.

Here is the caller graph for this function:



5.3.2 Variable Documentation

5.3.2.1 batch_size

```
int train.batch_size = 8
```

5.3.2.2 device

```
train.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

5.3.2.3 exist_ok

```
train.exist_ok
```

5.3.2.4 filename

```
train.filename
```

5.3.2.5 format

```
train.format
```

5.3.2.6 learning_rate

```
int train.learning_rate = 1e-3
```

5.3.2.7 level

```
train.level
```

5.3.2.8 LOAD_MODEL

```
bool train.LOAD_MODEL = False
```

5.3.2.9 num_epochs

```
int train.num_epochs = 1
```

5.3.2.10 patience

```
int train.patience = 5
```

5.3.2.11 save_dir

```
train.save_dir = 'checkpoints'
```

5.3.2.12 saved_images_dir

```
train.saved_images_dir = 'saved_images'
```

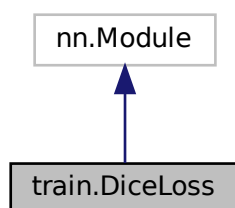

Chapter 6

Class Documentation

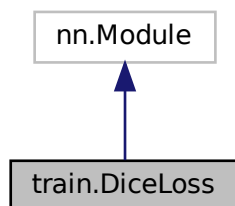
6.1 train.DiceLoss Class Reference

Dice Loss for multiclass segmentation.

Inheritance diagram for train.DiceLoss:



Collaboration diagram for train.DiceLoss:



Public Member Functions

- `def __init__` (self, `smooth`=1e-8)
- `def forward` (self, inputs, targets)

Forward pass for Dice Loss.

Public Attributes

- `smooth`

6.1.1 Detailed Description

Dice Loss for multiclass segmentation.

Measures the overlap between predicted and ground truth segmentation maps.

Parameters

<code>smooth</code>	(float, optional): Smoothing factor to avoid division by zero (default: 1e-8).
---------------------	--

6.1.2 Constructor & Destructor Documentation

6.1.2.1 `__init__()`

```
def train.DiceLoss.__init__ (  
    self,  
    smooth = 1e-8 )
```

6.1.3 Member Function Documentation

6.1.3.1 `forward()`

```
def train.DiceLoss.forward (  
    self,  
    inputs,  
    targets )
```

Forward pass for Dice Loss.

Parameters

<i>inputs</i>	(torch.Tensor): Model predictions [batch_size, num_classes, height, width].
<i>targets</i>	(torch.Tensor): Ground truth labels [batch_size, height, width].

Returns

torch.Tensor: Computed Dice loss.

6.1.4 Member Data Documentation

6.1.4.1 smooth

```
train.DiceLoss.smooth
```

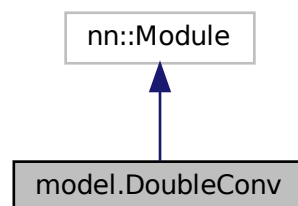
The documentation for this class was generated from the following file:

- [train.py](#)

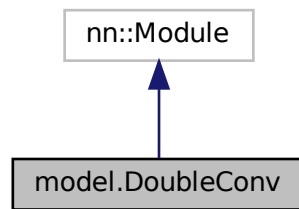
6.2 model.DoubleConv Class Reference

A double convolution block for the U-Net architecture.

Inheritance diagram for model.DoubleConv:



Collaboration diagram for model.DoubleConv:



Public Member Functions

- `def \_\_init\_\_ (self, in_channels, out_channels, dropout_rate=0.1)`
- `def forward (self, x)`
Forward pass for the double convolution block.

Public Attributes

- `conv`

6.2.1 Detailed Description

A double convolution block for the U-Net architecture.

This block applies two consecutive convolutional layers, each followed by batch normalization, ReLU activation, and a dropout layer to prevent overfitting.

Parameters

<i>in_channels</i>	(int): Number of input channels.
<i>out_channels</i>	(int): Number of output channels.
<i>dropout_rate</i>	(float, optional): Dropout probability (default: 0.1).

6.2.2 Constructor & Destructor Documentation

6.2.2.1 `__init__()`

```
def model.DoubleConv.__init__ (
    self,
```

```
in_channels,  
out_channels,  
dropout_rate = 0.1 )
```

6.2.3 Member Function Documentation

6.2.3.1 forward()

```
def model.DoubleConv.forward (   
    self,   
    x )
```

Forward pass for the double convolution block.

Parameters

x	(torch.Tensor): Input tensor of shape [batch_size, in_channels, height, width].
---	---

Returns

torch.Tensor: Output tensor after applying double convolution.

6.2.4 Member Data Documentation

6.2.4.1 conv

```
model.DoubleConv.conv
```

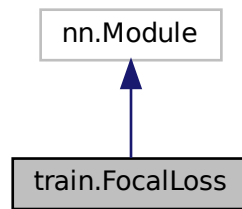
The documentation for this class was generated from the following file:

- [model.py](#)

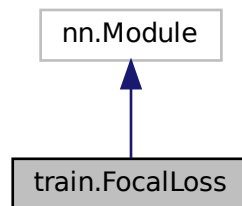
6.3 train.FocalLoss Class Reference

Focal Loss for multiclass segmentation.

Inheritance diagram for `train.FocalLoss`:



Collaboration diagram for `train.FocalLoss`:



Public Member Functions

- `def __init__` (self, `alpha`=0.95, `gamma`=2.0, `reduction`='mean')
- `def forward` (self, inputs, targets)

Forward pass for Focal Loss.

Public Attributes

- `alpha`
- `gamma`
- `reduction`

6.3.1 Detailed Description

Focal Loss for multiclass segmentation.

Focuses on hard-to-classify examples with a fixed alpha weight for foreground classes.

Parameters

<i>alpha</i>	(float, optional): Weight for foreground classes (default: 0.95).
<i>gamma</i>	(float, optional): Focusing parameter (default: 2.0).
<i>reduction</i>	(str, optional): Reduction method ('mean' or 'sum', default: 'mean').

6.3.2 Constructor & Destructor Documentation

6.3.2.1 __init__()

```
def train.FocalLoss.__init__ (
    self,
    alpha = 0.95,
    gamma = 2.0,
    reduction = 'mean' )
```

6.3.3 Member Function Documentation

6.3.3.1 forward()

```
def train.FocalLoss.forward (
    self,
    inputs,
    targets )
```

Forward pass for Focal Loss.

Parameters

<i>inputs</i>	(torch.Tensor): Model predictions [batch_size, num_classes, height, width].
<i>targets</i>	(torch.Tensor): Ground truth labels [batch_size, height, width].

Returns

torch.Tensor: Computed focal loss.

6.3.4 Member Data Documentation

6.3.4.1 alpha

`train.FocalLoss.alpha`

6.3.4.2 gamma

`train.FocalLoss.gamma`

6.3.4.3 reduction

`train.FocalLoss.reduction`

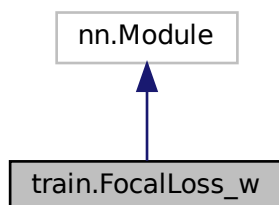
The documentation for this class was generated from the following file:

- [train.py](#)

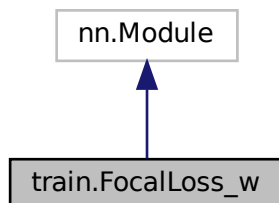
6.4 train.FocalLoss_w Class Reference

Focal Loss with class weights for multiclass segmentation.

Inheritance diagram for `train.FocalLoss_w`:



Collaboration diagram for `train.FocalLoss_w`:



Public Member Functions

- `def __init__` (self, `gamma`=2.0, `alpha`=None, `reduction`='mean')
- `def forward` (self, inputs, targets)

Forward pass for Focal Loss with weights.

Public Attributes

- `gamma`
- `alpha`
- `reduction`

6.4.1 Detailed Description

Focal Loss with class weights for multiclass segmentation.

Adjusts the loss to focus on hard-to-classify examples and balance class importance using weights.

Parameters

<i>gamma</i>	(float, optional): Focusing parameter (default: 2.0).
<i>alpha</i>	(torch.Tensor, optional): Class weights for balancing (default: None).
<i>reduction</i>	(str, optional): Reduction method ('mean' or 'sum', default: 'mean').

6.4.2 Constructor & Destructor Documentation

6.4.2.1 __init__()

```
def train.FocalLoss_w.__init__ (
    self,
    gamma = 2.0,
    alpha = None,
    reduction = 'mean' )
```

6.4.3 Member Function Documentation

6.4.3.1 forward()

```
def train.FocalLoss_w.forward (
    self,
    inputs,
    targets )
```

Forward pass for Focal Loss with weights.

Parameters

<i>inputs</i>	(torch.Tensor): Model predictions [batch_size, num_classes, height, width].
<i>targets</i>	(torch.Tensor): Ground truth labels [batch_size, height, width].

Returns

torch.Tensor: Computed focal loss.

6.4.4 Member Data Documentation

6.4.4.1 alpha

`train.FocalLoss_w.alpha`

6.4.4.2 gamma

`train.FocalLoss_w.gamma`

6.4.4.3 reduction

`train.FocalLoss_w.reduction`

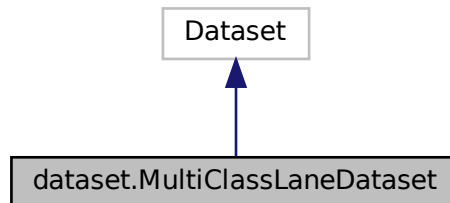
The documentation for this class was generated from the following file:

- [train.py](#)

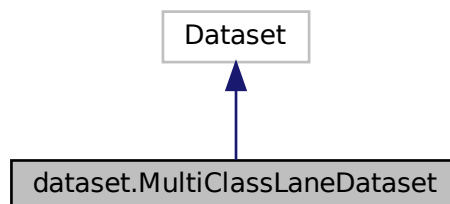
6.5 dataset.MultiClassLaneDataset Class Reference

A custom dataset for multiclass lane detection.

Inheritance diagram for dataset.MultiClassLaneDataset:



Collaboration diagram for dataset.MultiClassLaneDataset:



Public Member Functions

- `def __init__(self, image_dir, mask_dir, transform=None, num_augmentations=2)`
- `def __len__(self)`
Returns the total number of samples in the dataset.
- `def __getitem__(self, index)`
Retrieves an image and its corresponding multiclass mask by index.

Public Attributes

- `image_dir`
- `mask_dir`
- `transform`
- `num_augmentations`
- `images`
- `total_samples`
- `base_transform`

6.5.1 Detailed Description

A custom dataset for multiclass lane detection.

This class loads images and their corresponding multiclass masks (0: background, 1: driveable, 2: lanes), applies optional data augmentations, and returns them as PyTorch tensors for training or validation.

Parameters

<i>image_dir</i>	(str): Directory containing the input images.
<i>mask_dir</i>	(str): Directory containing the ground truth multiclass masks.
<i>transform</i>	(<code>albumentations.Compose</code> , optional): Data augmentation pipeline (default: None).
<i>num_augmentations</i>	(int, optional): Number of augmented versions per image (default: 2).

6.5.2 Constructor & Destructor Documentation

6.5.2.1 `__init__()`

```
def dataset.MultiClassLaneDataset.__init__ (
    self,
    image_dir,
    mask_dir,
    transform = None,
    num_augmentations = 2 )
```

6.5.3 Member Function Documentation

6.5.3.1 `__getitem__()`

```
def dataset.MultiClassLaneDataset.__getitem__ (
    self,
    index )
```

Retrieves an image and its corresponding multiclass mask by index.

Loads an image and mask, applies augmentations if specified, and returns them as tensors.

Parameters

<i>index</i>	(int): Index of the sample to retrieve.
--------------	---

Returns

tuple: A tuple containing the image tensor ([C, H, W]) and mask tensor ([H, W]).

6.5.3.2 __len__()

```
def dataset.MultiClassLaneDataset.__len__ (
    self )
```

Returns the total number of samples in the dataset.

Returns

int: Total number of samples, including augmentations.

6.5.4 Member Data Documentation**6.5.4.1 base_transform**

```
dataset.MultiClassLaneDataset.base_transform
```

6.5.4.2 image_dir

```
dataset.MultiClassLaneDataset.image_dir
```

6.5.4.3 images

```
dataset.MultiClassLaneDataset.images
```

6.5.4.4 mask_dir

```
dataset.MultiClassLaneDataset.mask_dir
```

6.5.4.5 num_augmentations

`dataset.MultiClassLaneDataset.num_augmentations`

6.5.4.6 total_samples

`dataset.MultiClassLaneDataset.total_samples`

6.5.4.7 transform

`dataset.MultiClassLaneDataset.transform`

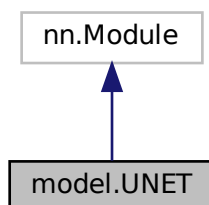
The documentation for this class was generated from the following file:

- [dataset.py](#)

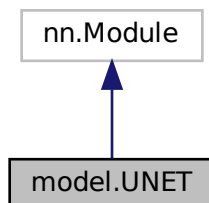
6.6 model.UNET Class Reference

U-Net architecture for semantic segmentation.

Inheritance diagram for model.UNET:



Collaboration diagram for model.UNET:



Public Member Functions

- def `__init__` (self, in_channels=3, out_channels=3, features=[32, 64, 128, 256])
- def `forward` (self, x)
Forward pass for the U-Net model.
- def `initialize_weights` (self)
Initializes weights for Conv2d, ConvTranspose2d, and BatchNorm2d layers.

Public Attributes

- `ups`
- `downs`
- `pool`
- `bottleneck`
- `final_conv`

6.6.1 Detailed Description

U-Net architecture for semantic segmentation.

Implements a U-Net with an encoder-decoder structure, including skip connections, for multiclass segmentation tasks. The model consists of a downsampling path, a bottleneck, and an upsampling path, with a final convolution to produce the output segmentation map.

Parameters

<i>in_channels</i>	(int, optional): Number of input channels (default: 3 for RGB images).
<i>out_channels</i>	(int, optional): Number of output channels/classes (default: 3 for multiclass segmentation).
<i>features</i>	(list, optional): List of feature sizes for each level (default: [32, 64, 128, 256]).

6.6.2 Constructor & Destructor Documentation

6.6.2.1 `__init__()`

```
def model.UNET.__init__ (
    self,
    in_channels = 3,
    out_channels = 3,
    features = [32, 64, 128, 256] )
```

6.6.3 Member Function Documentation

6.6.3.1 forward()

```
def model.UNET.forward (
    self,
    x )
```

Forward pass for the U-Net model.

Processes the input through the downsampling path, bottleneck, and upsampling path with skip connections, producing a segmentation map.

Parameters

<i>x</i>	(torch.Tensor): Input tensor of shape [batch_size, in_channels, height, width].
----------	---

Returns

torch.Tensor: Output segmentation map of shape [batch_size, out_channels, height, width].

6.6.3.2 initialize_weights()

```
def model.UNET.initialize_weights (
    self )
```

Initializes weights for Conv2d, ConvTranspose2d, and BatchNorm2d layers.

Uses Kaiming uniform initialization for convolutional layers and constant initialization for batch normalization.

6.6.4 Member Data Documentation

6.6.4.1 bottleneck

```
model.UNET.bottleneck
```

6.6.4.2 downs

```
model.UNET.downs
```


6.6.4.3 final_conv

`model.UNET.final_conv`

6.6.4.4 pool

`model.UNET.pool`

6.6.4.5 ups

`model.UNET.ups`

The documentation for this class was generated from the following file:

- [model.py](#)

Chapter 7

File Documentation

7.1 dataset.py File Reference

Classes

- class [dataset.MultiClassLaneDataset](#)
A custom dataset for multiclass lane detection.

Namespaces

- [dataset](#)

Variables

- [dataset.train_transforms](#)

7.2 model.py File Reference

Classes

- class [model.DoubleConv](#)
A double convolution block for the U-Net architecture.
- class [model.UNET](#)
U-Net architecture for semantic segmentation.

Namespaces

- [model](#)

7.3 train.py File Reference

Classes

- class [train.FocalLoss_w](#)
Focal Loss with class weights for multiclass segmentation.
- class [train.FocalLoss](#)
Focal Loss for multiclass segmentation.
- class [train.DiceLoss](#)
Dice Loss for multiclass segmentation.

Namespaces

- [train](#)

Functions

- def [train.check_masks](#) (image_dir, mask_dir, dataset_type="train")
Checks for missing image-mask pairs in the dataset.
- def [train.save_predictions_as_imgs](#) (loader, model, epoch, folder="saved_images/", device="cuda")
Saves model predictions as images alongside input images and ground truth masks.
- def [train.train_fn](#) (loader, model, optimizer, loss_fn_focal, loss_fn_dice, scaler, epoch)
Trains the model for one epoch.
- def [train.check_accuracy](#) (loader, model, loss_fn_focal, loss_fn_dice, device)
Evaluates the model on the validation dataset.
- def [train.load_checkpoint](#) (checkpoint, model)
Loads a saved model checkpoint.
- def [train.main](#) ()
Main training function.

Variables

- [train.device](#) = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
- int [train.batch_size](#) = 8
- int [train.num_epochs](#) = 1
- int [train.learning_rate](#) = 1e-3
- string [train.save_dir](#) = 'checkpoints'
- string [train.saved_images_dir](#) = 'saved_images'
- bool [train.LOAD_MODEL](#) = False
- int [train.patience](#) = 5
- [train.exist_ok](#)
- [train.filename](#)
- [train.level](#)
- [train.format](#)

Index

- `__getitem__`
 - `dataset.MultiClassLaneDataset`, 30
 - `__init__`
 - `dataset.MultiClassLaneDataset`, 30
 - `model.DoubleConv`, 22
 - `model.UNET`, 33
 - `train.DiceLoss`, 20
 - `train.FocalLoss`, 25
 - `train.FocalLoss_w`, 27
 - `__len__`
 - `dataset.MultiClassLaneDataset`, 31
- `alpha`
 - `train.FocalLoss`, 25
 - `train.FocalLoss_w`, 28
- `base_transform`
 - `dataset.MultiClassLaneDataset`, 31
- `batch_size`
 - `train`, 16
- `bottleneck`
 - `model.UNET`, 34
- `check_accuracy`
 - `train`, 10
- `check_masks`
 - `train`, 12
- `conv`
 - `model.DoubleConv`, 23
- `dataset`, 9
 - `train_transforms`, 9
- `dataset.MultiClassLaneDataset`, 29
 - `__getitem__`, 30
 - `__init__`, 30
 - `__len__`, 31
 - `base_transform`, 31
 - `image_dir`, 31
 - `images`, 31
 - `mask_dir`, 31
 - `num_augmentations`, 31
 - `total_samples`, 32
 - `transform`, 32
- `dataset.py`, 37
- `device`
 - `train`, 16
- `downs`
 - `model.UNET`, 34
- `exist_ok`
 - `train`, 16
- `filename`
 - `train`, 16
- `final_conv`
 - `model.UNET`, 34
- `format`
 - `train`, 16
- `forward`
 - `model.DoubleConv`, 23
 - `model.UNET`, 33
 - `train.DiceLoss`, 20
 - `train.FocalLoss`, 25
 - `train.FocalLoss_w`, 27
- `gamma`
 - `train.FocalLoss`, 26
 - `train.FocalLoss_w`, 28
- `image_dir`
 - `dataset.MultiClassLaneDataset`, 31
- `images`
 - `dataset.MultiClassLaneDataset`, 31
- `initialize_weights`
 - `model.UNET`, 34
- `learning_rate`
 - `train`, 16
- `level`
 - `train`, 17
- `load_checkpoint`
 - `train`, 13
- `LOAD_MODEL`
 - `train`, 17
- `main`
 - `train`, 13
- `mask_dir`
 - `dataset.MultiClassLaneDataset`, 31
- `model`, 9
 - `model.DoubleConv`, 21
 - `__init__`, 22
 - `conv`, 23
 - `forward`, 23
 - `model.py`, 37
 - `model.UNET`, 32
 - `__init__`, 33
 - `bottleneck`, 34
 - `downs`, 34
 - `final_conv`, 34
 - `forward`, 33
 - `initialize_weights`, 34

- pool, 35
- ups, 35
- num_augmentations
 - dataset.MultiClassLaneDataset, 31
- num_epochs
 - train, 17
- patience
 - train, 17
- pool
 - model.UNET, 35
- reduction
 - train.FocalLoss, 26
 - train.FocalLoss_w, 28
- save_dir
 - train, 17
- save_predictions_as_imgs
 - train, 14
- saved_images_dir
 - train, 17
- smooth
 - train.DiceLoss, 21
- total_samples
 - dataset.MultiClassLaneDataset, 32
- train, 10
 - batch_size, 16
 - check_accuracy, 10
 - check_masks, 12
 - device, 16
 - exist_ok, 16
 - filename, 16
 - format, 16
 - learning_rate, 16
 - level, 17
 - load_checkpoint, 13
 - LOAD_MODEL, 17
 - main, 13
 - num_epochs, 17
 - patience, 17
 - save_dir, 17
 - save_predictions_as_imgs, 14
 - saved_images_dir, 17
 - train_fn, 15
- train.DiceLoss, 19
 - __init__, 20
 - forward, 20
 - smooth, 21
- train.FocalLoss, 23
 - __init__, 25
 - alpha, 25
 - forward, 25
 - gamma, 26
 - reduction, 26
- train.FocalLoss_w, 26
 - __init__, 27
 - alpha, 28
 - forward, 27
 - gamma, 28
 - reduction, 28
- train.py, 38
- train_fn
 - train, 15
- train_transforms
 - dataset, 9
- transform
 - dataset.MultiClassLaneDataset, 32
- ups
 - model.UNET, 35